PRNGs and Diffie-Hellman Key Exchange

CSE 405 January 2026
Lecture 6

Last Time: Hashes

- Map arbitrary-length input to fixed-length output
- Output is deterministic and unpredictable
- Security properties
  - One way: Given an output y, it is infeasible to find any input x such that $H(x) = y$.
  - Collision resistant: It is infeasible to find another any pair of inputs $x' \neq x$ such that $H(x) = H(x')$.
- Some hashes are vulnerable to length extension attacks
- Hashes don't provide integrity (unless you can publish the hash securely)

Last Time: MACs

- Inputs: a secret key and a message
- Output: a tag on the message
- A secure MAC is unforgeable: Even if Mallory can trick Alice into creating MACs for messages that Mallory chooses, Mallory cannot create a valid MAC on a message that she hasn't seen before
- Example: HMAC(K, M) = $H((K' \oplus opad) \,||\, H((K' \oplus ipad) \,||\, M))$
- MACs do not provide confidentiality

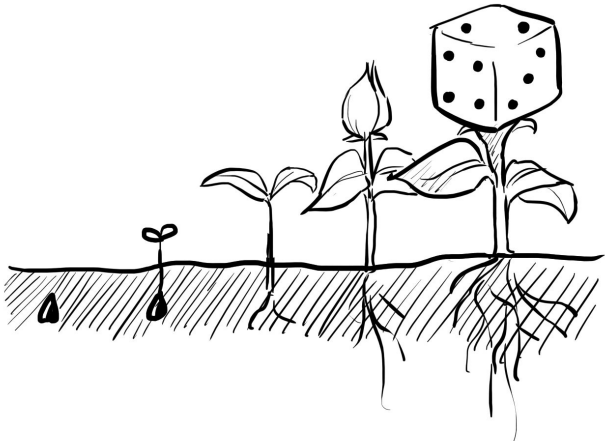
Last Time: Authenticated Encryption

- Authenticated encryption: A scheme that simultaneously guarantees confidentiality and integrity (and authenticity) on a message
- First approach: Combine schemes that provide confidentiality with schemes that provide integrity and authenticity
  - MAC-then-encrypt: Enc(K_1, M || MAC(K_2, M))
  - Encrypt-then-MAC: Enc(K_1, M) || MAC(K_2, Enc(K_1, M))
  - Always use Encrypt-then-MAC because it's more robust to mistakes
- Second approach: Use AEAD encryption modes designed to provide confidentiality, integrity, and authenticity
  - Drawback: Incorrectly using AEAD modes leads to losing *both* confidentiality and integrity/authentication

Today: PRNGs and Diffie-Hellman Key Exchange

- Symmetric-key encryption schemes need randomness. How do we securely generate random numbers?
- When discussing symmetric-key schemes, we assumed Alice and Bob managed to share a secret key. How can Alice and Bob share a symmetric key over an insecure channel?

Pseudorandom Number Generators (PRNGs)



Textbook Chapter 9

Cryptography Roadmap

	Symmetric-key	Asymmetric-key
Confidentiality	• One-time pads • Block ciphers with chaining modes (e.g. AES-CBC)	• RSA encryption • ElGamal encryption
Integrity, Authentication	• MACs (e.g. HMAC)	• Digital signatures (e.g. RSA signatures)

- Hash functions
- Pseudorandom number generators
- Public key exchange (e.g. Diffie-Hellman)
- Key management (certificates)
- Password management

Randomness

- Randomness is essential for symmetric-key encryption
  - A random key
  - A random IV/nonce
  - Universally unique identifiers (we'll see this shortly)
  - We'll see more applications later
- If an attacker can predict a random number, things can catastrophically fail
  - Think about NID?
  - Birth Registration Number?
  - How should you assign ID in your 408 project?
- How do we securely generate random numbers?

Entropy

- In cryptography, “random” usually means “random and unpredictable”
- Scenario
  - You want to generate a secret bitstring that the attacker can't guess
  - You generate random bits by tossing a fair (50-50) coin
  - The outcomes of the fair coin are harder for the attacker to guess
- **Entropy**: A measure of uncertainty
  - In other words, a measure of how unpredictable the outcomes are
  - High entropy = unpredictable outcomes = desirable in cryptography
  - The uniform distribution has the highest entropy (every outcome equally likely, e.g. fair coin toss)
  - Usually measured in bits (so 3 bits of entropy = uniform, random distribution over 8 values)

Breaking Bitcoin Wallets

- What happens if we use a poor source of entropy?
- Bitcoin users use a randomly-generated private key to access their account (and money)
  - An attacker who learns the key can access the money
  - We'll learn more about Bitcoin later
- An "improvment" [sic] to the algorithm reduced the entropy used to generate the private keys
  - Any private key created with this "improvment" could be brute-forced



Improvments to RNG

[redacted] committed on Dec 7, 2014 1 parent b0d5639

Showing 1 changed file with 26 additions and 28 deletions.

54 bitcoinjs-lib/src/jsbn/rng.js

```
@@ -8,15 +8,16 @@ var rng_state;
 8  8  var rng_pool;
 9  9  var rng_pptr;
10 10
11    - // Mix in a 32-bit integer into the pool
12    - function rng_seed_int(x) {
13    -     rng_pool[rng_pptr++] ^= x & 255;
14    -     rng_pool[rng_pptr++] ^= (x >> 8) & 255;
15    -     rng_pool[rng_pptr++] ^= (x >> 16) & 255;
16    -     rng_pool[rng_pptr++] ^= (x >> 24) & 255;
```

XZ Utils Backdoor

- One of the most sophisticated supply chain attacks ever discovered in open-source software

CVE-2024-3094 | CVSS: 10.0

https://www.youtube.com/watch?v=F7iLfuci75Y

True Randomness

- To generate truly random numbers, we need a physical source of entropy
  - An unpredictable circuit on a CPU
  - Human activity measured at very fine time scales (e.g. the microsecond you pressed a key)
- Unbiased entropy usually requires combining multiple entropy sources
  - Goal: Make the total randomness equal the sum of each source's entropy.
    - Many poor sources + 1 good source = good entropy
- Issues with true randomness
  - It's expensive and slow to generate
  - Physical entropy sources are often biased



Exotic entropy source: Cloudflare has a wall of lava lamps that are recorded by an HD video camera that views the lamps through a rotating prism

The Lava Lamps That Help Keep The Internet Secure

Pseudorandom Number Generators (PRNGs)

- True randomness is expensive
- **Pseudorandom number generator** (**PRNGs**): An algorithm that uses a little bit of true randomness (seed) to generate a lot of random-looking output
  - Also called **deterministic random bit generators** (**DRBGs**)
- Usage
  - Generate some expensive true randomness (e.g. noisy circuit on your CPU)
  - Use the true randomness as input to the PRNG
  - Generate random-looking numbers quickly and cheaply with the PRNG
- PRNGs are deterministic: Output is generated according to a set algorithm
  - However, for an attacker who can't see the internal state, the output is *computationally indistinguishable* from true randomness

PRNG: Definition

- A PRNG has three functions:
  - PRNG.Seed(randomness): Initializes the internal state using the entropy
    - Input: Some truly random bits
  - PRNG.Reseed(randomness): Updates the internal state using the existing state and the entropy
    - Input: More truly random bits
  - PRNG.Generate(*n*): Generate *n* pseudorandom bits
    - Input: A number *n*
    - Output: *n* pseudorandom bits
    - Updates the internal state as needed
- Properties
  - **Correctness**: Deterministic
  - **Efficiency**: Efficient to generate pseudorandom bits
  - **Security**: Indistinguishability from random
  - **Additional security**: Rollback resistance

PRNG: Seeding and Reseeding

- A PRNG should be seeded with all available sources of entropy
  - Combining many low-entropy sources should result in high-entropy output
  - If one source has 0 entropy (constant), it should not reduce the entropy of the output
- Reseeding is used to add even more entropy as it becomes available
  - Reseeding with 0 entropy should not reduce the entropy of the internal state or output

PRNG: Security

- Can we design a PRNG that is truly random?
- A PRNG cannot be truly random
  - The output is deterministic given the initial seed
  - If the initial seed is *s* bits long, there are only 2^s possible output sequences
  - Function: $\{0, 1\}^S \rightarrow \{0, 1\}^{2S}$ has 2^S inputs and 2^S outputs even though the output space is 2^{2S}
- A secure PRNG is computationally indistinguishable from random to an attacker
  - Game: Present an attacker with a truly random sequence and a sequence outputted from a secure PRNG
  - An attacker should not be able to determine which is which with probability > 1/2 + negligible
- Equivalent definition: An attacker cannot predict future output of the PRNG

Insecure PRNGs: Breaking Slot Machines

- What happens if PRNGs are used improperly?
- A casino in St. Louis experienced unusual bad “luck”
  - Suspicious players would hover over the lever and then spin at a specific time to win
- Vulnerability: Slot machines used predictable PRNGs
  - The PRNG output was based on the current time
- Strategy:
  - Use a smartphone to alert you to when to pull the lever for the best chance of winning
- Las Vegas was not affected by the vulnerability
  - Nevada slot machines must follow evaluation standards designed to address this sort of issue

Insecure PRNGs: OpenSSL PRNG bug

- What happens if we don't use enough entropy?
- Debian OpenSSL CVE-2008-0166
  - Debian: A Linux distribution
  - OpenSSL: A cryptographic library
  - In "cleaning up" OpenSSL (Debian "bug" #363516), the author "fixed" how OpenSSL seeds random numbers
  - The existing code caused Purify and Valgrind to complain about reading uninitialized memory
  - The cleanup caused the PRNG to only be seeded with the process ID
  - There are only 2^{15} (32,768) possible process IDs, so the PRNG only has 15 bits of entropy
- Easy to deduce private keys generated with the PRNG
  - Set the PRNG to every possible starting state and generate a few private/public key pairs
  - See if the matching public key is anywhere on the Internet



PRNG: Rollback Resistance

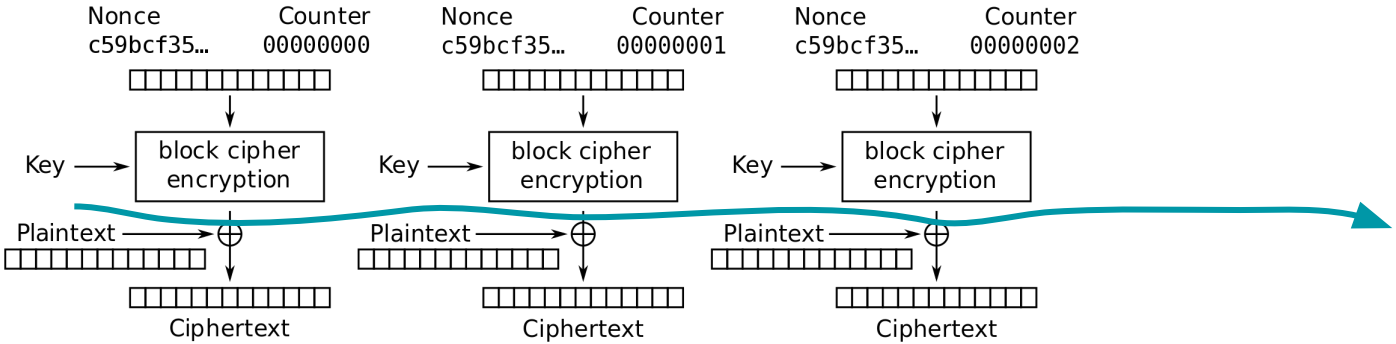
- **Rollback resistance**: If the attacker learns the internal PRNG state (e.g., by hacking into the machine and dumping its memory), they cannot learn anything about previous states or outputs
  - Game: An attacker knows the current internal state of the PRNG and is given a sequence of truly random bits and a sequence of previous output from the PRNG
  - The attacker cannot determine which is which with probability > 1/2
- Rollback resistance is not required in a secure PRNG, but it is a useful property
  - Consider:
    - Alice uses the same PRNG to generate her secret key and the IVs for encryption
    - Mallory compromises the internal state of the PRNG
    - If the PRNG is not rollback resistant, Mallory can derive previous PRNG output… such as the secret key

PRNG: Rollback Resistance

- **Rollback resistance**: If the attacker learns the internal PRNG state (e.g., by hacking into the machine and dumping its memory), they cannot learn anything about previous states or outputs
  - Also called
    - Backtracking resistance
    - Forward secrecy
  - Every time output is generated, the internal state updates using a hash function, block cipher, or HMAC, for example: $State_{T+1}=HMAC(State_T)$
  - Reversing the state requires finding a hash preimage or inverting a block cipher without the key, which is computationally infeasible.

CTR-DRBG (Deterministic Random Bit Generator)

- AES in CTR mode generates m pseudo random bits by applying AES m/128 times and concatenating the result
- Generate(m) = $E_k(IV \| 1) \| E_k(IV \| 2) \| E_k(IV \| 3) \ldots E_k(IV \| ceil(m/128))$



Counter (CTR) mode encryption

HMAC-DRBG

- Idea: HMAC output looks unpredictable. Let's use HMAC to build a PRNG!
- HMAC takes two arguments (key and message). Let's keep two values, *K* (key) and *V* (value) as internal state
- To generate a block of pseudorandom bits, compute HMAC on the previous block of output.
    - This can be repeated to generate as many pseudorandom bits as needed.

HMAC-DRBG

Seed(*s*):

K := 0

V := 0

Reseed(s)



Reseed(s):

K := HMAC(*K*, *V* || 0x00 || *s*)

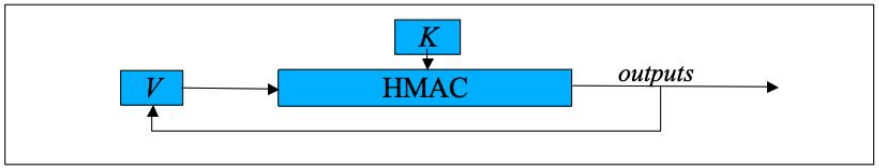
V := HMAC(*K*, *V*)

K := HMAC(*K*, *V* || 0x01 || *s*)

V := HMAC(*K*, *V*)



HMAC-DRBG



Generate(*n*):

output = ''
while len(output) < *n* **do**
    V = HMAC(*K*, *V*)
    output = output || *V*
end while

K = HMAC(*K*, *V* || 0x00)
V = HMAC(*K*, *V*)

return output[:*n*]

Call HMAC repeatedly to generate random-looking output

Update internal state with no extra entropy

HMAC-DRBG: Security

- Assuming HMAC is secure, HMAC-DRBG is a secure, rollback-resistant PRNG
  - Secure: If you can distinguish PRNG output from random, then you've distinguished HMAC from random
  - Rollback-resistant: If you can derive old output from the current state, then you've reversed the hash function or HMAC
  - The full proof is out of scope
  - In other words: if you break HMAC-DRBG, you've either broken HMAC or the underlying hash function

Insecure PRNGs: CVE-2019-16303

- Relevant if you wrote an app in JHipster before 2019
- Password reset functions
  - When you forget your password, receive an email with a special link to reset your password
  - The special link should contain a randomly-generated code (so attackers can't make their own link)
- Vulnerability: Bad PRNG
  - You can figure out the PRNG’s internal state from the reset link
  - Request password reset links for other people's accounts
  - Predict the “random” reset link and take over any account you want!

Insecure PRNGs: Rust Rand_Core

- A Rust library has an interface for “secure” random number generators… but it isn’t actually secure!
- Example: ChaCha8Rng
  - A stream cipher PRNG
  - No reseed function: no way of adding extra entropy after the initial seed
  - Seed only takes 32 bits: no way to combine entropy
  - No rollback resistance
- None of the “secure” RNGs are cryptographically secure
  - None have a reseed function to add extra entropy
  - None take arbitrarily long seeds
- **Takeaway**: Always make sure you use a secure PRNG
  - Consider human factors? Use fail-safe defaults?

Application: Universally Unique Identifiers (UUIDs)

- Scenario
  - You have a set of objects (e.g. files)
  - You need to assign a unique name to every object
  - Every name must be unique and unpredictable
- Solution: Choose a random value
  - If you use enough randomness, the probability of generating the same random value twice are astronomically small (basically 0)
    - It's more likely that your hardware will fail or that a cosmic ray will flip a bit in your machine's memory than it is that you'll experience a UUID collision.
- Universally Unique Identifiers (UUIDs)
  - 128-bit unique values
  - To generate a new UUID, seed a secure PRNG properly, and generate a random value
  - Often written in hexadecimal: `00112233-4455-6677-8899-aabbccddeeff`
  - You'll work with UUIDs in 408 Project

Stream Ciphers

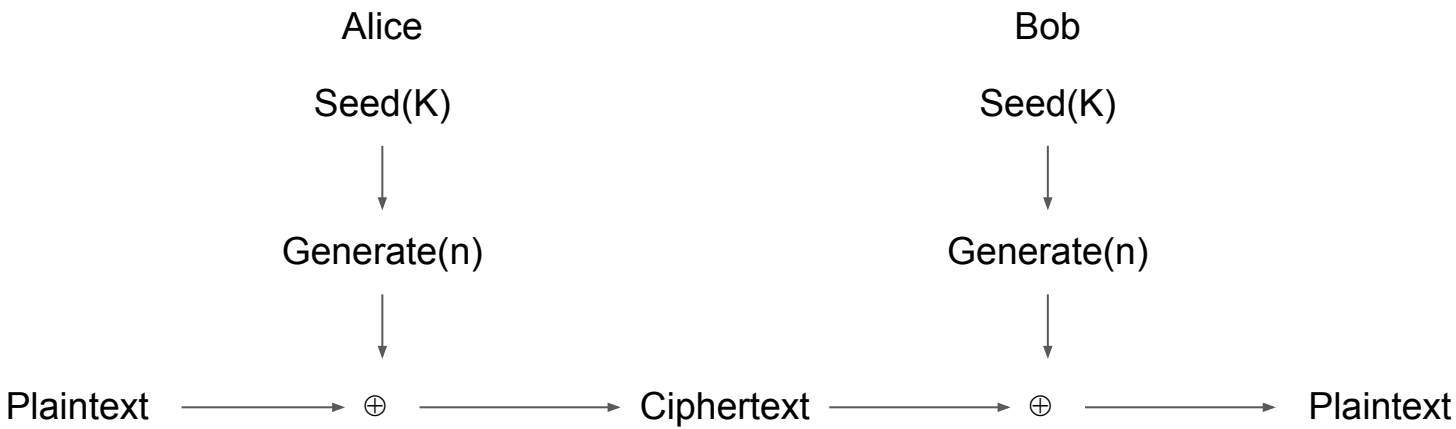
Textbook Chapter 9.5

Stream Ciphers

- Another way to construct symmetric key encryption schemes
- Idea
  - A secure PRNG produces output that looks indistinguishable from random
  - An attacker who can't see the internal PRNG state can't learn any output
  - What if we used PRNG output as the key to a one-time pad?
- **Stream cipher**: A symmetric encryption algorithm that uses pseudorandom bits as the key to a one-time pad

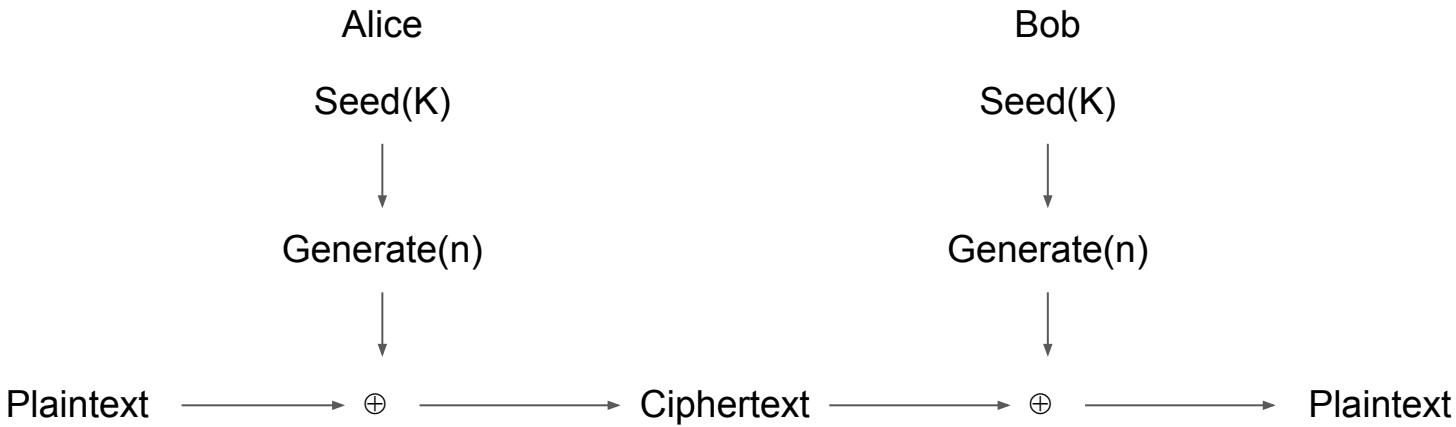
Stream Ciphers

- Protocol: Alice and Bob both seed a secure PRNG with their symmetric secret key, and then use the output as the key for a one-time pad



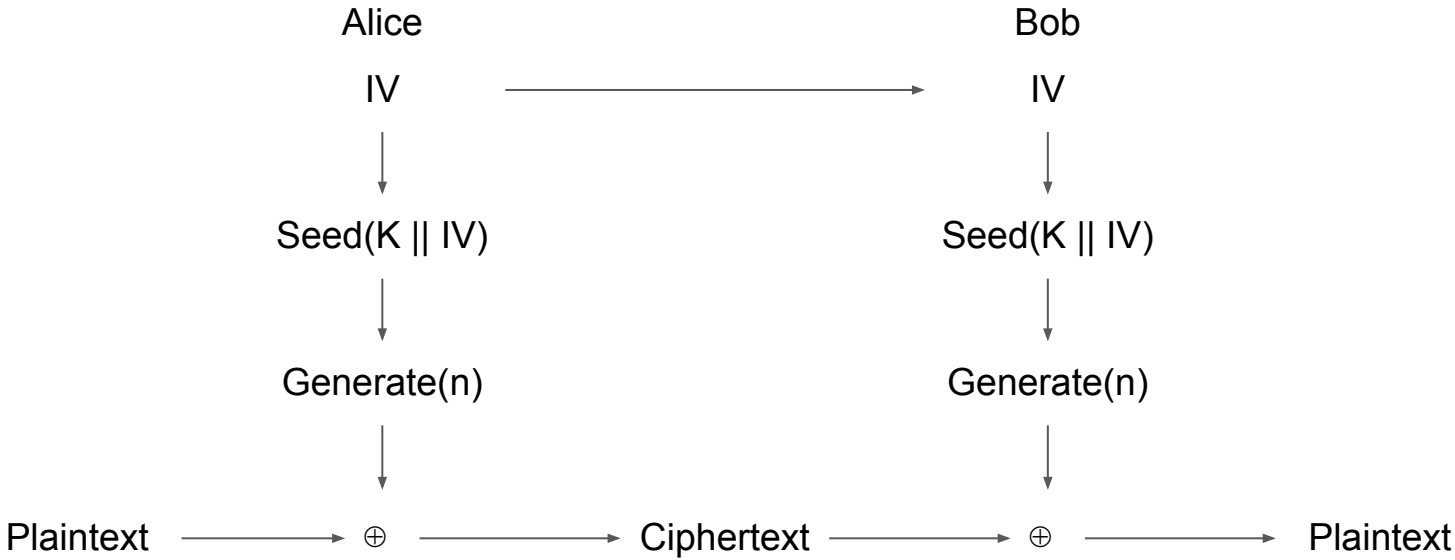
Stream Ciphers: Encrypting Multiple Messages

- Recall: One-time pads are insecure when the key is reused. How do we encrypt multiple messages without key reuse?



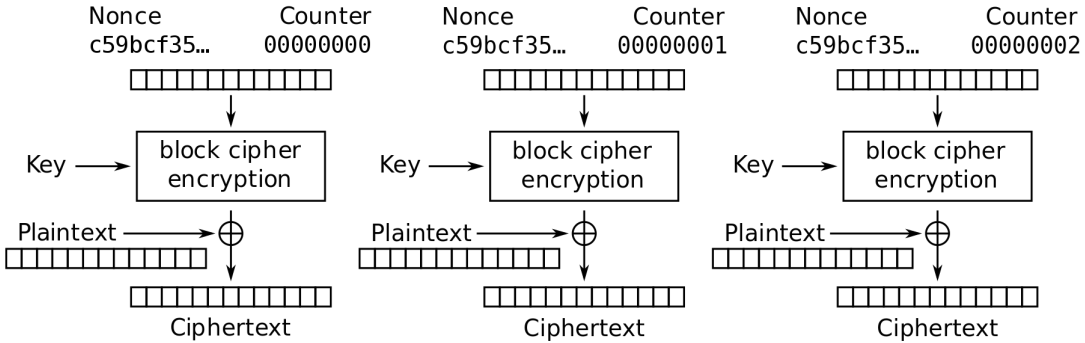
Stream Ciphers: Encrypting Multiple Messages

- Solution: For each message, seed the PRNG with the key and a random IV, concatenated. Send the IV with the ciphertext



Stream Ciphers: AES-CTR

- If you squint carefully, AES-CTR is a type of stream cipher
- Output of the block cipher is pseudorandom and used as a one-time pad



Counter (CTR) mode encryption

Stream Ciphers: Security

- Stream ciphers are IND-CPA secure, assuming the pseudorandom output is secure
- In some stream ciphers, security is compromised if too much plaintext is encrypted
  - Example: In AES-CTR, if you encrypt so many blocks that the counter wraps around, you'll start reusing keys
  - In practice, if the key is n bits long, usually stop after $2^{n/2}$ bits of output
  - Example: In AES-CTR with 128-bit counters, stop after 2^{64} blocks of output

Stream Ciphers Support Streaming

- Stream ciphers can continually process new data as it arrives
  - Only need to maintain internal state of the PRNG
  - Keep generating more PRNG output as more input arrives

Stream Ciphers Support Seeking

- Suppose you received a 1 GB ciphertext (encryption of a 1 GB message) and you only wanted to decrypt the last 128 bytes
- Benefit of some stream ciphers: You can decrypt one part of the ciphertext without decrypting the entire ciphertext
  - Example: In AES-CTR, to decrypt only block *i*, compute E_K(nonce || *i*) and XOR with the *i*th block of ciphertext
  - Example: ChaCha20 (another stream cipher) lets you decrypt arbitrary parts of ciphertext
  - With HMAC-DRBG, you have to generate all the PRNG output up until the block you want to decrypt

Next: Diffie-Hellman Key Exchange

- When discussing symmetric-key schemes, we assumed Alice and Bob managed to share a secret key. How can Alice and Bob share a symmetric key over an insecure channel?

Diffie-Hellman Key Exchange

Textbook Chapter 10

Cryptography Roadmap

	Symmetric-key	Asymmetric-key
Confidentiality	• One-time pads • Block ciphers with chaining modes (e.g. AES-CBC)	• RSA encryption • ElGamal encryption
Integrity, Authentication	• MACs (e.g. HMAC)	• Digital signatures (e.g. RSA signatures)

- Hash functions
- Pseudorandom number generators
- Public key exchange (e.g. Diffie-Hellman)
- Key management (certificates)
- Password management

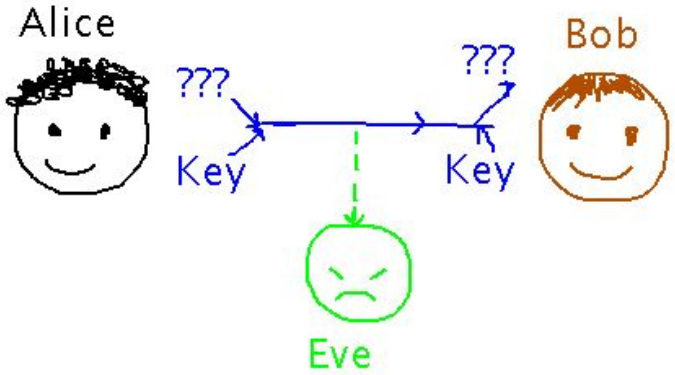
Diffie-Hellman Key Exchange

https://www.youtube.com/watch?v=YEBfamv-_do



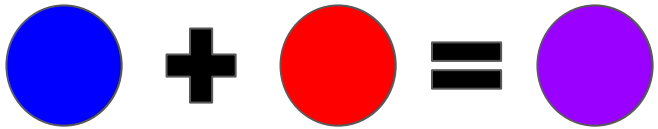
Sharing Secret Keys

- Eve is always listening
- How would Alice and Bob share a secret key through an insecure channel?



Mixing Colors!

- Suppose the key is a secret color.
- Mixing colors is a one-way function

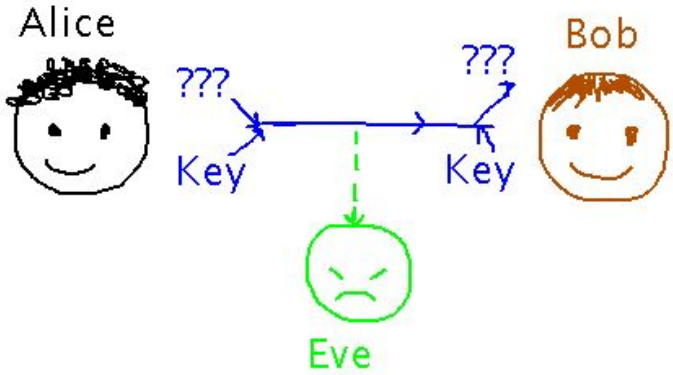




Easy

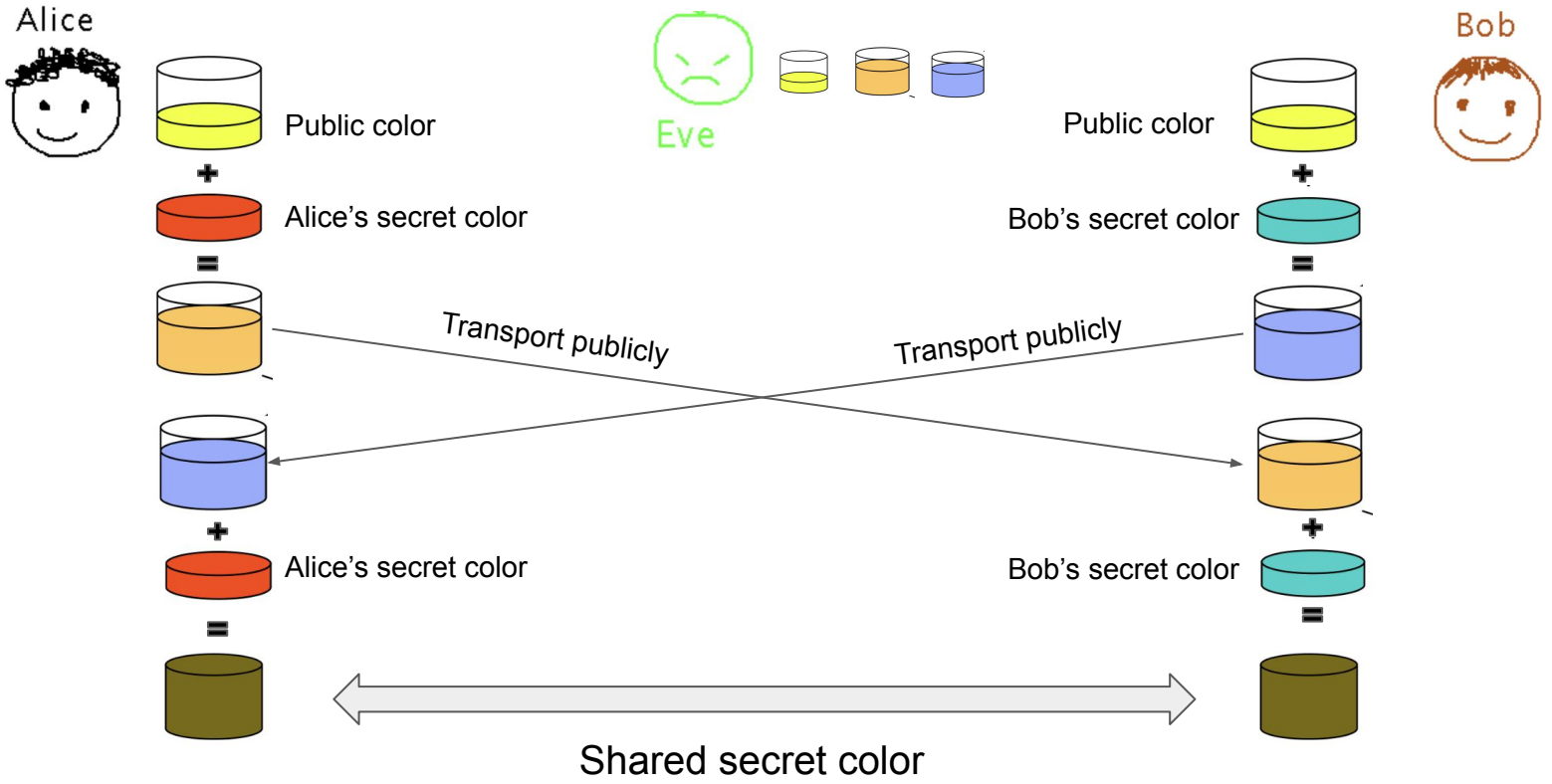


Hard



Mathematical equivalent of color mixing: Discrete logarithm problem

Secure Color Sharing



Secure Color Sharing

- Suppose Alice and Bob want a secret *paint color*, but Eve can see paint colors sent between Alice and Bob
- Alice generates a secret color **amber A**, and Bob generates a secret color **blue B**
- Alice and Bob agree on a common, public color **green G**
- They both mix their secret colors with **G**, so Alice has **green-amber GA**, and Bob has **green-blue GB**
- Alice sends **GA** to Bob, and Bob sends **GB** to Alice
  - Note: Eve now knows the colors **GA** and **GB**! Assume that it is hard to separate colors.
- Alice knows **GB**, so she can mix in **A** to form green-amber-blue **GAB**. Bob knows **GA**, so he can mix in **B** to form **GAB**, as well!
  - Eve only knows **G**, **GA**, and **GB**, so she can only form **green-amber-green-blue GAGB**, which is not the same!



Discrete Log Problem and Diffie-Hellman Problem

- There is a mathematical version of this
- Assume everyone knows a large prime p (e.g. 2048 bits long) and a number g
- **Discrete logarithm problem** (**discrete log problem**): Given g, p, $g^a \bmod p$ for random a, it is computationally hard to find a
- **Diffie-Hellman assumption**: Given g, p, $g^a \bmod p$, and $g^b \bmod p$ for random a, b, no polynomial time attacker can distinguish between a random value R and $g^{ab} \bmod p$.
  - Intuition: The best known algorithm is to first calculate a and then compute $(g^b)^a \bmod p$, but this requires solving the discrete log problem, which is hard!
  - Note: Multiplying the values doesn't work, since you get $g^{a+b} \bmod p \neq g^{ab} \bmod p$

Diffie-Hellman Key Exchange



Alice

Public: g, p

Eve



Bob



Secret key → Generate a

Generate b

Calculate $g^a \bmod p$

g^a

g^b

Calculate $g^b \bmod p$

Public key

Receive $g^b \bmod p$

Receive $g^a \bmod p$

Calculate $(g^b)^a \bmod p$

Calculate $(g^a)^b \bmod p$

$a, g^a, g^b \Rightarrow g^{ab}$

$g^a, g^b \not\Rightarrow g^{ab}$

$b, g^a, g^b \Rightarrow g^{ab}$

Shared symmetric key is g^{ab}

Discrete Log Problem and Diffie-Hellman Problem

- g, p are publicly shared
- a, b are secret to Alice and Bob. Only Alice knows a, and only Bob knows b.
- g^a mod p, and g^b mod p ⇒ these too are publicly shared.
- g is called the generator (also called primitive root mod p)
  - g lies within [2, p-1]
  - g^a mod p distributes uniformly over [1, p-1] for all possible choices of a.
    - In the context of number theory, if g is a primitive root of p, then g^a mod p has residues [1, p-1] and it has a period of p-1.
  - All possible choices of “a” includes: [0, p-2]. As soon as “a” becomes p-1, the modulo starts repeating from 1.
    - According to Euler’s theorem, $g^{\varphi(p)} \equiv 1 \bmod p$. Since p is a prime number, $\varphi(p) = p-1$.

Discrete Log Problem and Diffie-Hellman Problem

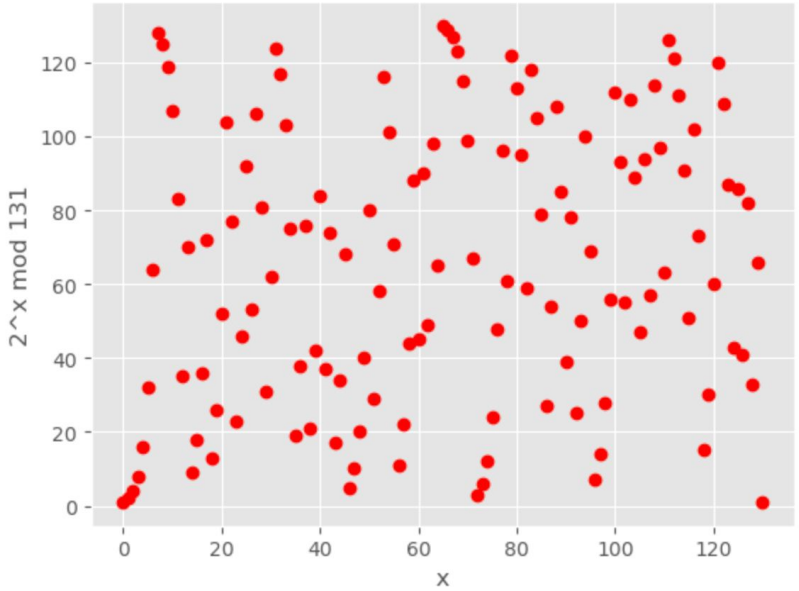
In Diffie–Hellman key exchange, the values a and b (private keys) must satisfy these properties:

1. **Randomly chosen**: They should be chosen randomly and kept secret.
2. **In the range**: $1 \leq a, b \leq p - 2$, where p is a large prime.
3. **Relatively prime not required**: They do **not** need to be coprime to $p - 1$.
4. **Sufficiently large**: To ensure security, a and b should be large integers (hundreds or thousands of bits in real applications).

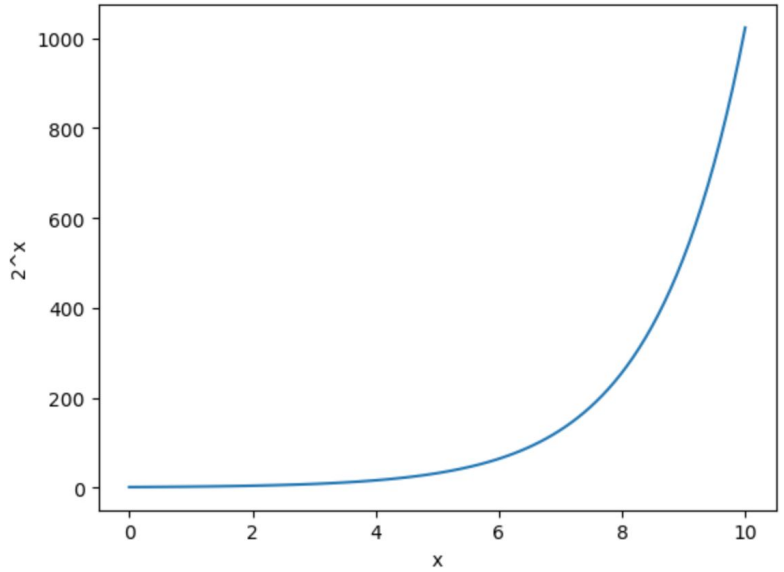
These private keys are then used to compute public keys:

- $A = g^a \mod p$
- $B = g^b \mod p$

Discrete Log Problem



Plot of g^a mod p



Plot of g^a

https://www.desmos.com/calculator/3j0h11za2s

Why must g be a primitive root?

- If g isn't a generator (primitive root), then
  - g^{ab} mod p only covers a subset of [1,2,... p-1]
  - The shared secret g^{ab} mod p lives in that smaller subgroup, so it has less entropy
  - Less entropy = more predictable = easier to break / brute force.

Ephemerality of Diffie-Hellman

- Diffie-Hellman can be used ephemerally (called Diffie-Hellman ephemeral, or DHE)
  - **Ephemeral**: Short-term and temporary, not permanent
  - Alice picks a fresh random a, Bob picks a fresh random b for each session.
  - They derive their shared secret $K=g^{ab} \bmod p$
  - Once the session ends, they delete a, b and K.
  - Even if Eve steals Alice and Bob's secrets somehow later on, those values are going to be different, so Eve cannot decrypt any of the past messages.

Diffie-Hellman: Man In the Middle Attack



Alice

Mallory



Public: g, p

Bob



Generate a

Calculate $g^a \bmod p$

Generate m

Calculate $g^m \bmod p$

Generate b

Calculate $g^b \bmod p$

Receive $g^m \bmod p$

Calculate $(g^m)^a \bmod p$

a, g^a, $g^m \Rightarrow g^{am}$

Receive $g^a \bmod p$

Calculate $(g^a)^m \bmod p$

m, g^m, $g^a \Rightarrow g^{am}$

Receive $g^b \bmod p$

Calculate $(g^b)^m \bmod p$

m, g^m, $g^b \Rightarrow g^{bm}$

Receive $g^m \bmod p$

Calculate $(g^m)^b \bmod p$

b, g^b, $g^m \Rightarrow g^{bm}$

Diffie-Hellman: Issues

- Diffie-Hellman is not secure against a MITM adversary
- DHE is an *active protocol*: Alice and Bob need to be online at the same time to exchange keys
  - What if Bob wants to encrypt something and send it to Alice for her to read later?
  - Next time: How do we use *public-key encryption* to send encrypted messages when Alice and Bob don't share keys and aren't online at the same time?
- Diffie-Hellman does not provide *authentication*
  - You exchanged keys with someone, but Diffie-Hellman makes no guarantees about who you exchanged keys with; it could be Mallory!

Diffie-Hellman: Exercise

(c) Explain the philosophy behind Diffie-Hellman key exchange protocol. In the Diffie-Hellman key exchange protocol, let the prime number be 353 and one of its prime root is 3. Let the users A and B select their secret keys 97 and 233, respectively. Compute the public keys of A and B and also their common secret key. **(3+7=10)**

https://www.khanacademy.org/computing/computer-science/cryptography/modarithmetic/a/fast-modular-exponentiation

Group

A **group** $(G, \star)$ is a set G plus a binary operation $\star$ satisfying:

1. **Closure:** $a, b \in G \Rightarrow a \star b \in G$
2. **Associativity:** $(a \star b) \star c = a \star (b \star c)$
3. **Identity:** $\exists\, e \in G$ with $e \star a = a \star e = a$ for all a
4. **Inverses:** $\forall\, a \in G$, $\exists\, a^{-1} \in G$ with $a \star a^{-1} = a^{-1} \star a = e$.

Elliptic-Curve Diffie-Hellman (ECDH)

- Notice: The discrete-log problem seems hard because exponentiating integers in modular arithmetic “wraps around”
  - Diffie-Hellman can be generalized to any mathematical group that has this cyclic property
  - Discrete-log uses the “multiplicative group of integers mod *p* under generator *g*”
- Elliptic curves: A type of mathematical curve
  - Big idea: Repeatedly adding a point to itself on a curve is another cyclic group
  - You don’t need to understand the math behind elliptic curves
- **Elliptic-curve Diffie-Hellman**: A variation of Diffie-Hellman that uses elliptic curves instead of modular arithmetic
  - Based on the elliptic curve discrete log problem, the analog of the discrete log problem
  - Benefit of ECDH: The underlying problem is harder to solve, so we can use smaller keys (3072-bit DHE is about as secure as 384-bit ECDHE)

Summary: Diffie-Hellman Key Exchange

- Algorithm:
  - Alice chooses a and sends $g^a \bmod p$ to Bob
  - Bob chooses b and sends $g^b \bmod p$ to Alice
  - Their shared secret is $(g^a)^b = (g^b)^a = g^{ab} \bmod p$
- Diffie-Hellman provides forwards secrecy: Nothing is saved or can be recorded that can ever recover the key
- Diffie-Hellman can be performed over other mathematical groups, such as elliptic-curve Diffie-Hellman (ECDH)
- Issues
  - *Not* secure against MITM
  - Both parties must be online
  - Does not provide authenticity